

Discrimination/Classification

Introduction

In **classification** we wish to predict the value of a categorical variable based on the values of a set of predictors.

Consider the Smarket set data set, which gives daily percentage returns for the S&P 500 stock index between 2001 and 2005.

```
library(ISLR)
attach(Smarket)
head(Smarket)
```

##	Year	Lag1	Lag2	Lag3	Lag4	Lag5	Volume	Today	Direction
## 1	2001	0.381	-0.192	-2.624	-1.055	5.010	1.1913	0.959	Up
## 2	2001	0.959	0.381	-0.192	-2.624	-1.055	1.2965	1.032	Up
## 3	2001	1.032	0.959	0.381	-0.192	-2.624	1.4112	-0.623	Down
## 4	2001	-0.623	1.032	0.959	0.381	-0.192	1.2760	0.614	Up
## 5	2001	0.614	-0.623	1.032	0.959	0.381	1.2057	0.213	Up
## 6	2001	0.213	0.614	-0.623	1.032	0.959	1.3491	1.392	Up

The variables are

Year The year that the observation was recorded

Lag1 Percentage return for previous day

Lag2 Percentage return for 2 days previous

Lag3 Percentage return for 3 days previous

Lag4 Percentage return for 4 days previous

Lag5 Percentage return for 5 days previous

Volume Volume of shares traded (number of daily shares traded in billions)

Today Percentage return for today

Direction A factor with levels Down and Up indicating whether the market had a positive or negative return on a given day

Our goal is to predict the variable *Direction* based on previous days' performance. That is, we wish to predict whether the market will go up or down based on what did in recent days.

Set-up

Suppose that we have a training sample of input-output pairs (X_i, Y_i) , $i = 1, \dots, n$, where X_i are input variables taking values in a set $X \subset R^p$ and Y_i are output variables taking values in $\{1, \dots, K\}$. Suppose one observes a new input X . The problem is how one guess (predicts) the value of Y associated with the observed input X based on the training sample. It has two main goals:

1) *Discrimination*: Use the information in a learning set of labeled observations to construct a classifier (or classification rule) that will separate the predefined classes as much as possible. Which predictors are most important?

2) *Classification*: Given a set of measurements on a new unlabeled observation, use the classifier to predict the class of that observation. Sometimes it is called “supervised learning” or “class prediction”.

Discrimination is focused on finding the features (i.e. predictors) that divide a set of known objects into groups. *Classification* is focused on finding a rule that will accurately classify new into one of a set of groups.

Bayes Classifier

For simplicity, suppose that we have a single predictor X . Our goal is to predict the values of the categorical variable Y based on the value of X .

Take $f_k(x)$ as the density of x when the observation comes from the k^{th} class:

$$f_k(x) = P(X|Y = k)$$

Take π_k as the prior probability (that is, the probability without knowing X) that an observation belongs to the k^{th} class. Then, by Bayes theorem:

$$P(Y = k|X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^k \pi_l f_l(x)}$$

We will denote this probability as $p_k(x)$:

$$p_k(x) = P(Y = k|X = x)$$

Example

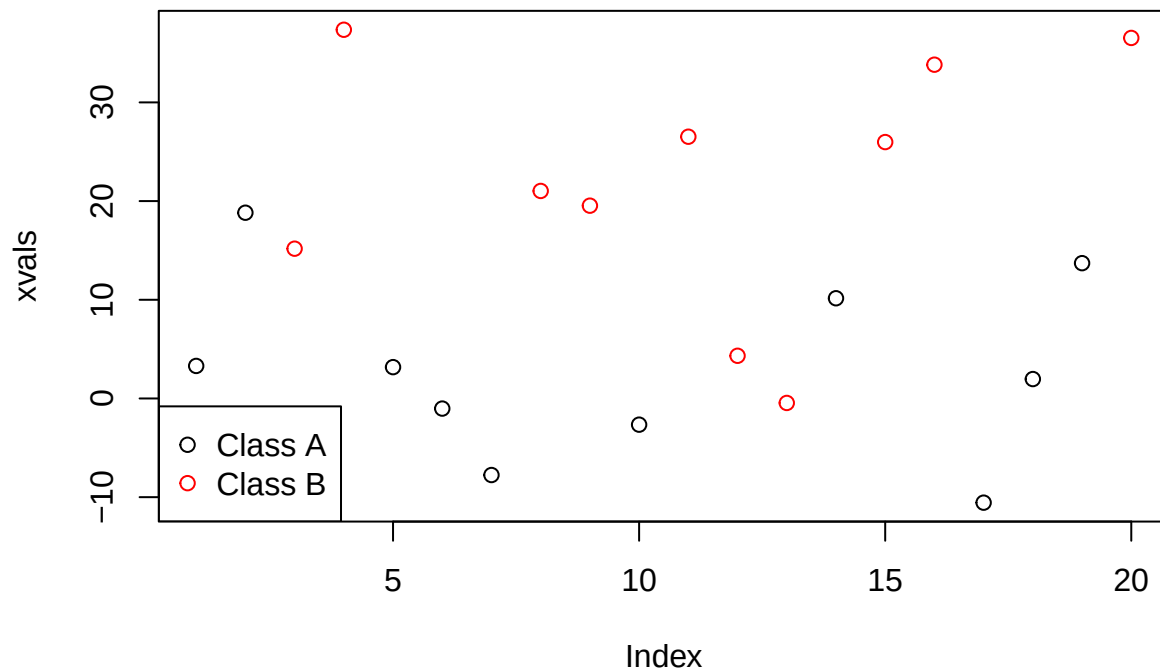
Consider an example with two classes A and B. If the observation is from class A, then X has $normal(0, 10)$ distribution. If the observation is from class B, then X has a $normal(20, 10)$ distribution. In the code below, I do the following:

1. randomly assign observation to the classes A and B with equal probability.
2. Assign the population mean for the observation based on the class.
3. Generate the x value for the observation based on the mean.
4. Make a plot of the x -values, with points colored black if they are from class A and red if they are from class B.

```
set.seed(1112)
n=20 #sample size
muA=0 #mean for class A
muB=20 # mean for class B
sd0=10 #standard deviation

classY=sample(c("A","B"),n,replace=T) #randomly assign class with equal probability
meanX=ifelse(classY=="A",muA,muB) #assign the population mean based on the class
xvals=rnorm(n,mean=meanX,sd0) #generate the x value based on the mean

#plot
plot(xvals,col=ifelse(classY=="A","black","red"))
legend("bottomleft",pch=1,c("Class A","Class B"),col=c("black","red"))
```



From this plot, we see that most observations from class A have $X < 10$ and most observations from class B have $X > 10$. Thus, for this data the following rule seems reasonable:

1. Classify the observation as A if $X < 10$
2. Classify the observation B if $X > 10$

This rule is not perfect - we will be wrong for some data points - but it will get the class for most observations correct.

Posterior probabilities For this example, we know that the two classes have equal probability of being in class A or B. Thus

$$\pi_A = \pi_B = 0.5$$

The density for X given the class Y is the normal distribution:

$$f_A(x) = \text{normal}(\mu = 0, \sigma = 10)$$

$$f_B(x) = \text{normal}(\mu = 20, \sigma = 10)$$

Thus, the posterior probabilities can be calculated as

$$P(Y = A|X = x) = \frac{\pi_A f_A(x)}{\pi_A f_A(x) + \pi_B f_B(x)}$$

The code below calculates this for each value of x in `xvals`:

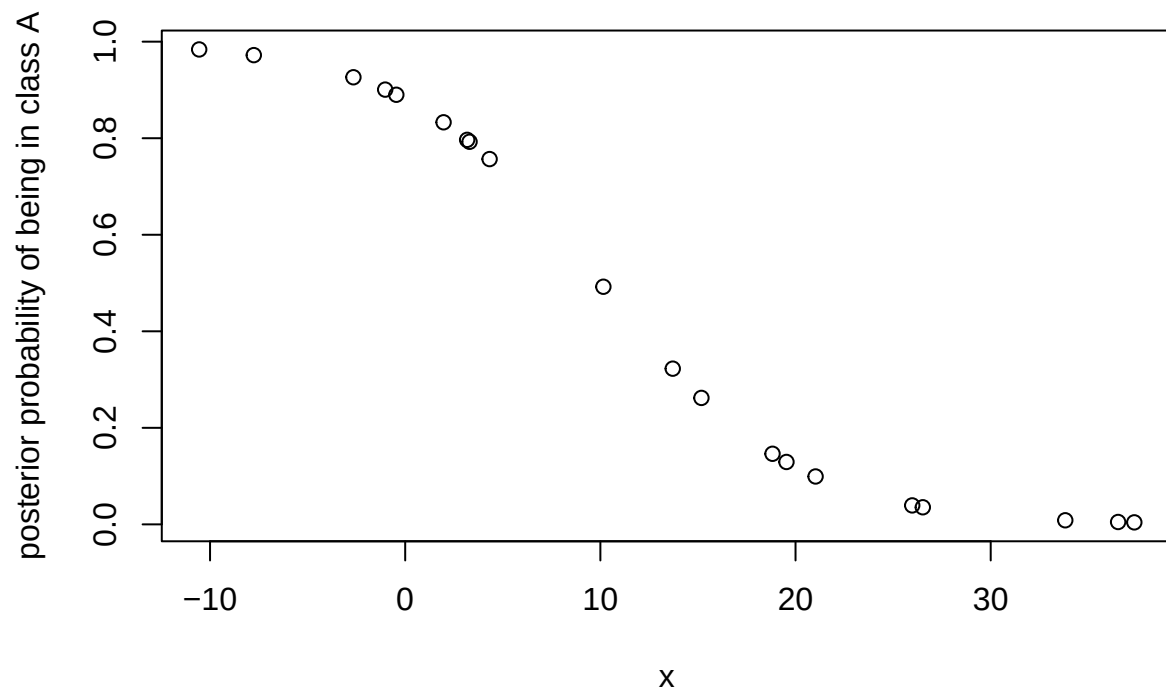
```
#posterior probability for being class A

postprobA=0.5*dnorm(xvals,mean=muA,sd=sd0)/(0.5*dnorm(xvals,mean=muA,sd=sd0)+0.5*dnorm(xvals,mean=muB,sd=sd0))

dd=data.frame(xvals,postprobA)
names(dd)=c("x","posterior probability")
dd[1:20,]
```

```
##           x posterior probability
## 1    3.2963296      0.792610634
## 2   18.8211815      0.146260567
## 3   15.1777315      0.262010251
## 4   37.3592798      0.004185829
## 5    3.1736532      0.796614771
## 6   -1.0252851      0.900702715
## 7   -7.7575471      0.972118367
## 8   21.0274223      0.099259063
## 9   19.5350345      0.129317485
## 10  -2.6541244      0.926274704
## 11  26.5224463      0.035417502
## 12   4.3241301      0.756792474
## 13  -0.4553443      0.890032084
## 14  10.1532781      0.492336695
## 15  25.9790119      0.039323992
## 16  33.8207224      0.008458034
## 17 -10.5563646      0.983877299
## 18   1.9654281      0.832982548
## 19  13.7091926      0.322602240
## 20  36.5296158      0.004937614
```

```
plot(dd[,1],dd[,2],xlab="x",ylab="posterior probability of being in class A")
```



In this example (which has large separation between the classes), the class probability has a strong dependence on X .

Gaussian-Based Methods

This example points us towards how we can calculate posterior probabilities. In the example, we know the prior class probabilities and the density function for X and so it is simple to calculate the posterior probabilities. In a real situation, we won't know these things. However, if we are willing to make an assumption about the form of $f_k(x)$, then we can estimate them.

If we have training data, then we can estimate the prior probabilities simply as the proportion of observations in each class. If n_k is the number of observations in the training data in class k and n is the total number of observations in the training data, then

$$\hat{\pi}_k = \frac{n_k}{n}$$

In Gaussian-based methods, we assume that $f_k(x)$ follows a normal distribution:

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} e^{-\frac{1}{2\sigma_k^2}(x-\mu_k)^2}$$

Then, the posterior density becomes

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi}\sigma_k} e^{-\frac{1}{2\sigma_k^2}(x-\mu_k)^2}}{\sum_{l=1}^K \pi_l \frac{1}{\sqrt{2\pi}\sigma_l} e^{-\frac{1}{2\sigma_l^2}(x-\mu_l)^2}}$$

Decision rule

We then use the following decision rule:

The predicted class of an observation with $X = x$ is the class with the highest $p_k(x)$

Thus, in order to find the predicted class for an observation x , we must find the value of k that maximizes the above expression.

The denominator does not depend on k . Thus, we can ignore it in our maximization. Taking $\log p_k(x)$ and ignoring the denominator, we get

$$\log p_k(x) = \log \pi_k - \log \sqrt{2\pi}\sigma_k - \frac{1}{2\sigma_k^2}(x - \mu_k)^2$$

For simplicity, we take $\sigma_k = \sigma \forall k$. Then, we get

$$\log p_k(x) = \log \pi_k - \log \sqrt{2\pi}\sigma - \frac{1}{2\sigma^2}(x^2 - 2\mu_k x + \mu_k^2)$$

The terms $\log \sqrt{2\pi}\sigma$ and x^2 do not depend on k and can be ignored. Thus, we are left with

$$\delta_k(x) = \log \pi_k + \frac{\mu_k x}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2}$$

Thus

predicted class for observation x is the k that maximizes $\delta_k(X)$

Exercise 1

In the previous example, find the predicted class for each observation. Make a table with rows corresponding to the predicted class and columns corresponding to the actual class and with counts for each category. Hint #1: Note that we know the mean and variance because we simulated the data. Use these in the calculation of $\delta_k(X)$. Hint #2: The `table` command is useful for making the table.

Repeat for `sd=40`.

The steps to do this are:

1. Generate the data using my code from above.
2. Calculate δ_A and δ_B .
3. Find the predicted class for each data point as the class corresponding to the higher delta.
4. Make the table of predicted and actual class using `table`.

Linear Discriminant Analysis

In the above exercise, we know the true means and variance. In a real data situation, we would have to estimate them from the training data:

$$\begin{aligned}\hat{\mu}_k &= \frac{1}{n_k} \sum_{i:y_i=k} x_i \\ \hat{\sigma}^2 &= \frac{1}{n-K} \sum_{k=1}^K \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2 \\ \hat{\pi}_k &= \frac{n_k}{n}\end{aligned}$$

If we plug these estimates into the above decision rule, we have **linear discriminant analysis**. It is called *linear* discriminant analysis because $\delta_k(x)$ is linear in x .

Exercise 2

In this exercise, we will find (train) the LDA classifier using the data set above and then use it to predict values in a new data set.

1. Use the data set from exercise 1 to estimate the parameters μ , σ , and π .
2. Write functions to calculate $\delta_A(x)$ and $\delta_B(x)$.
3. Make a new data set from the same distribution.
4. Use `ifelse` and your `delta` functions to predict the class for each new observation.
5. Produce a table similar to the previous exercise. Find the proportion of correct classifications.
6. Use the classifier from exercise 1 (i.e. the one with the true parameter values) and produce the table and the probability of correct classification.

Extending to multiple predictors

The LDA method extends to multiple predictors in a straightforward way. See the book section 4.4.2.

Other Gaussian-based methods

Constraints on covariance structure with multiple predictors lead to different forms of discriminant analysis:

- Nearest Centroid: $\Sigma_j = I_p$
- Diagonal Linear Discriminant Analysis: diagonal covariance matrix
- Fisher's Linear Discriminant Analysis: $\Sigma_1 = \dots = \Sigma_K$ (most popular)
- Quadratic Discriminant Analysis: allow different covariance matrices

Linear Discriminant Analysis in R

The `lda` function does linear discriminant analysis in R:

```
library(MASS)
library(ISLR)
attach(Smarket)
```

```
## The following objects are masked from Smarket (pos = 4):
##
##   Direction, Lag1, Lag2, Lag3, Lag4, Lag5, Today, Volume, Year
```

```
train =which(Year <2005)
```

```
lda.fit=lda(Direction~Lag1+Lag2,data=Smarket,subset=train)
lda.fit
```

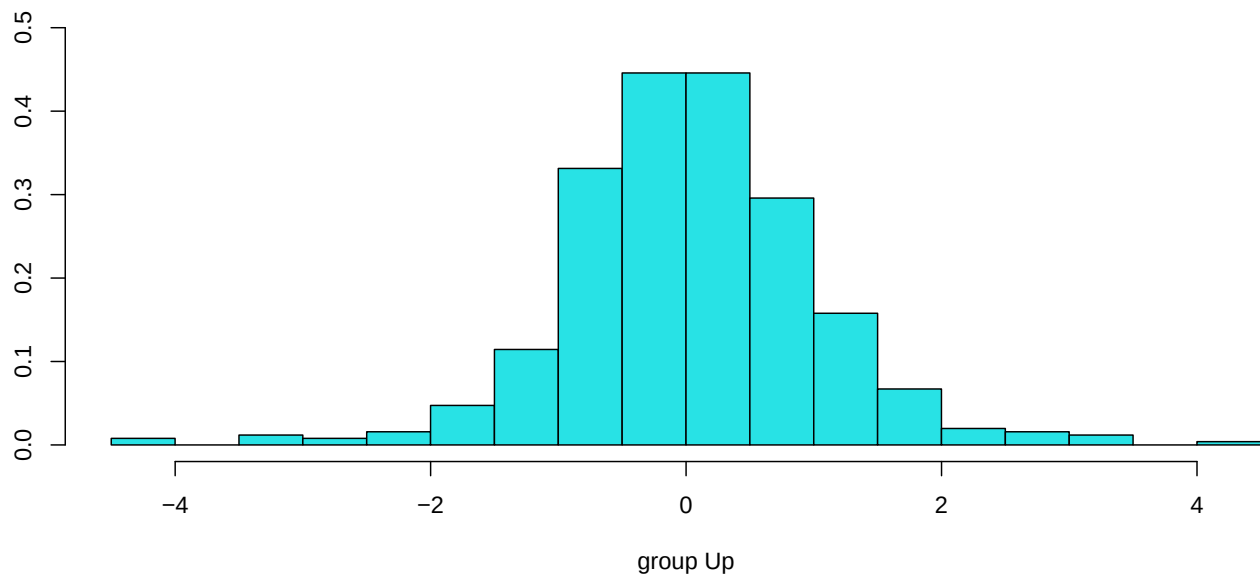
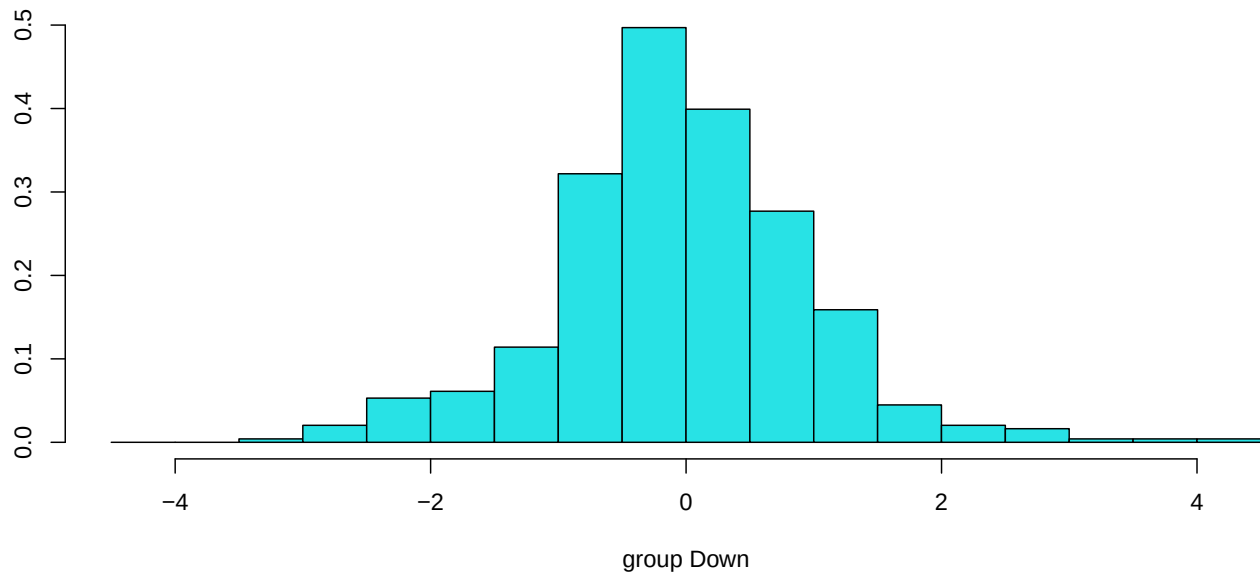
```
## Call:
## lda(Direction ~ Lag1 + Lag2, data = Smarket, subset = train)
##
## Prior probabilities of groups:
##      Down      Up
## 0.491984 0.508016
##
## Group means:
##      Lag1      Lag2
## Down 0.04279022 0.03389409
## Up   -0.03954635 -0.03132544
##
## Coefficients of linear discriminants:
##      LD1
## Lag1 -0.6420190
## Lag2 -0.5135293
```

Group means: These suggest that there is a tendency for the previous 2 days' returns to be negative on days when the market increases, and a tendency for the previous days' returns to be positive on days when the market declines.

Coefficients: The discrimination rule is given by the coefficients. if $-0.642 \times \text{Lag1} - 0.514 \times \text{Lag2}$ is large, then the LDA classifier will predict a market increase, and if it is small, then the LDA classifier will predict a market decline.

The plot shows the distribution of the values given by the discrimination rule for the two values of the response:

```
plot(lda.fit)
```



There is not much difference in the distribution.

leave-one-out error If we specify `CV=TRUE`, then `lda` does leave-one-out cross validation.

```
lda.fit=lda(Direction~Lag1+Lag2,data=Smarket,CV=TRUE)
```

It fits the model with all of the data except one point and then predicts the class for that point. The `class` component returns the predicted values:

```
lda.fit$class[1:30]
```

```
## [1] Up Down Down Up Up Up Down Up Up Down Down Up Up Up Down
```

```
## [16] Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up   Up
## Levels: Down Up
```

Let's compare predicted with actual:

```
table(lda.fit$class,Direction)
```

```
##      Direction
##      Down  Up
## Down  109 109
## Up    493 539
```

The columns are the actual direction that the stock market went. The rows are the predicted. Our discrimination rule does not do very well! When it predicted **Down**, it had an equal chance of being **Down** or **Up**. When it predicted **Up** there was only a small increase in the chance of being up.

The `posterior` component returns posterior probabilities for the each point using the model fit with the rest of the data:

```
lda.fit$posterior[1:30]
```

```
## [1] 0.4861599 0.5030997 0.5098580 0.4822116 0.4857059 0.4921773 0.5080112
## [8] 0.4903248 0.4775162 0.5055021 0.5004531 0.4756548 0.4726615 0.4916168
## [15] 0.5011796 0.4794415 0.4847526 0.4573986 0.4674574 0.4826875 0.4645767
## [22] 0.4606981 0.4515300 0.4867475 0.4784711 0.4682409 0.4932804 0.4554305
## [29] 0.4275850 0.4273132
```

These are consistent with the results above. We do not have much predictive ability with this data.

testing error Next, we will split the data into training and testing sets. We use the data from 2001-2004 as training data and predict 2005. We use the `predict.lda` function to make predictions for the testing data from the trained model:

```
train =(Year <2005)
lda.fit=lda(Direction~Lag1+Lag2,data=Smarket,subset=train)
Smarket.2005= Smarket[!train ,]
lda.pred=predict(lda.fit, Smarket.2005)
```

The components of `lda.predict` are the predicted class, the estimated posterior probability for the classes, and the lda scores of predicted data. We will again make a table of predicted versus actual values:

```
names(lda.pred) # assign class based on posterior probability
```

```
## [1] "class"      "posterior" "x"
```

```
lda.class=lda.pred$class
Direction.2005=Smarket.2005[, "Direction"]
table(lda.class,Direction.2005) #confusion matrix
```

```
##          Direction.2005
## lda.class Down  Up
##      Down   35  35
##      Up    76 106
```

```
mean(lda.class==Direction.2005) #classification accuracy
```

```
## [1] 0.5595238
```

```
sum(lda.pred$posterior[,1]>=.5)
```

```
## [1] 70
```

```
sum(lda.pred$posterior[,1]<.5)
```

```
## [1] 182
```

```
lda.pred$posterior[1:20,1]
```

```
##      999      1000      1001      1002      1003      1004      1005      1006
## 0.4901792 0.4792185 0.4668185 0.4740011 0.4927877 0.4938562 0.4951016 0.4872861
##      1007      1008      1009      1010      1011      1012      1013      1014
## 0.4907013 0.4844026 0.4906963 0.5119988 0.4895152 0.4706761 0.4744593 0.4799583
##      1015      1016      1017      1018
## 0.4935775 0.5030894 0.4978806 0.4886331
```

```
lda.class[1:20]
```

```
## [1] Up  Up  Up  Up  Up  Up  Up  Up  Up  Up  Up  Up  Down Up  Up  Up
## [16] Up  Up  Down Up  Up
## Levels: Down Up
```

```
sum(lda.pred$posterior[,1]>.9) # could use a different threshold
```

```
## [1] 0
```

Exercise #3 Rerun the above analysis using the years 2002-2005 as the training data and 2001 as the test data. Make the resulting confusion matrix and calculate test accuracy.

Quadratic Discriminant Analysis

The qda function is similar, but uses a quadratic rather than linear decision rule:

```
qda.fit=qda(Direction~Lag1+Lag2,data=Smarket,subset=train)
qda.class=predict(qda.fit,Smarket.2005)$class
table(qda.class,Direction.2005)
```

```
##          Direction.2005
## qda.class Down  Up
##      Down   30  20
##      Up    81 121
```

```
mean(qda.class==Direction.2005)
```

```
## [1] 0.5992063
```

It does slightly better than LDA.

k - Nearest Neighbor method

In the KNN method each object x is assigned to the class of majority of its k nearest neighbors.

KNN Procedure

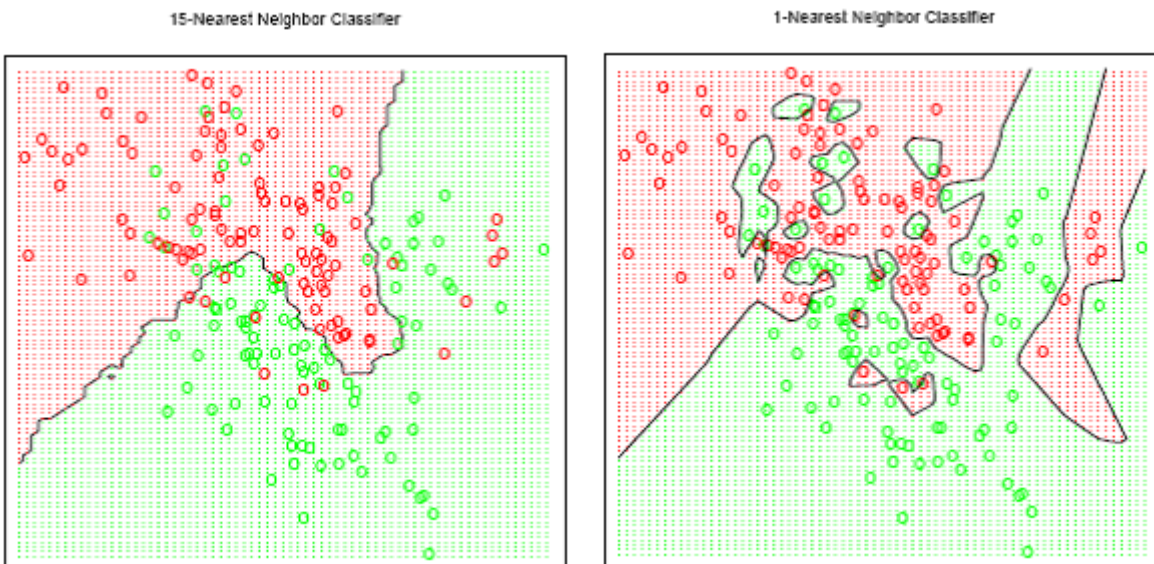
Suppose that we have a training sample of input-output pairs (X_i, Y_i) , $i = 1, \dots, n$, where X_i are input variables taking values in a set $X \subset R^p$ and Y_i are output variables taking values in $\{1, \dots, L\}$. Consider a test value X_t for which we wish to predict the output category Y_t .

We follow the steps below:

1. Calculate the distance $d(X_t, X_i)$, $i = 1..n$ between the new data point and each training point.
2. Find the K nearest points to X_t . That is, the K points with the smallest value of $d(X_t, X_i)$.
3. Find the most frequent value of Y among these K points. This is the predicted value of Y for the input X_t .
4. If there is a tie in the most frequent category Y , one of the tied values is chosen randomly.

Choice of K

When $k=1$, it is assigned to the class of its nearest neighbor and the separating boundary will be very irregular. A larger k usually yields smoother boundary.



The choice of K is a tradeoff between the risk of overfitting (fitting to random variation) and oversmoothing (ignoring true variation).

Example

We will start by making a simulated data set. We assume that the data follows a bivariate normal distribution with different mean and (potentially) variance for each of three categories:

```
library(MASS) #need for mvrnorm function

set.seed(1234)

#sample sizes for the three categories 1, 2, and 3:
n=50
n1=n
n2=n
n3=n

#means for the three categories
mu_x1=10
mu_y1=10
mu_x2=30
mu_y2=30
mu_x3=10
mu_y3=30

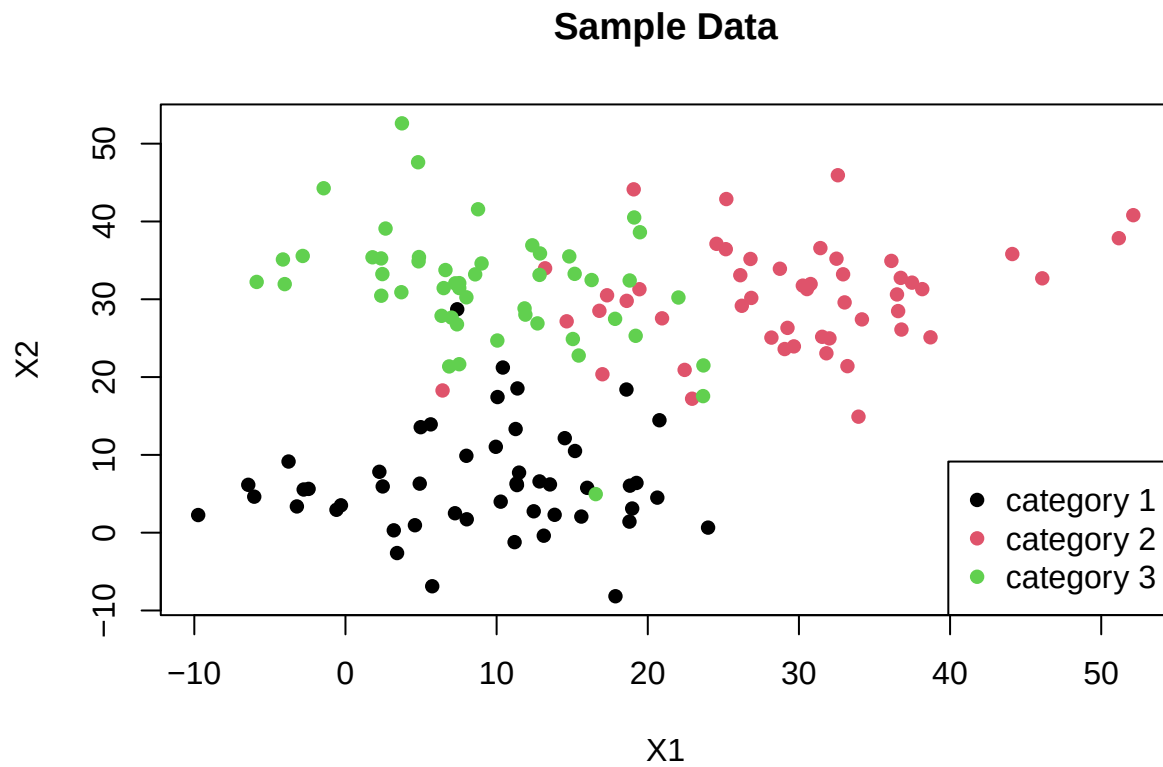
sd0=60 #SD

#generate values for sample 1:

d1=mvrnorm(n1,mu=c(mu_x1,mu_y1),Sigma=sd0*diag(2))
d2=mvrnorm(n2,mu=c(mu_x2,mu_y2),Sigma=sd0*diag(2))
d3=mvrnorm(n3,mu=c(mu_x3,mu_y3),Sigma=sd0*diag(2))

dd=rbind(d1,d2,d3)
dd=cbind(dd,c(rep(1,n1),rep(2,n2),rep(3,n3)))

plot(dd[,1:2],col=dd[,3],xlab="X1",ylab="X2",main="Sample Data",pch=16)
legend("bottomright",c("category 1","category 2","category 3"),col=1:3,pch=16)
```

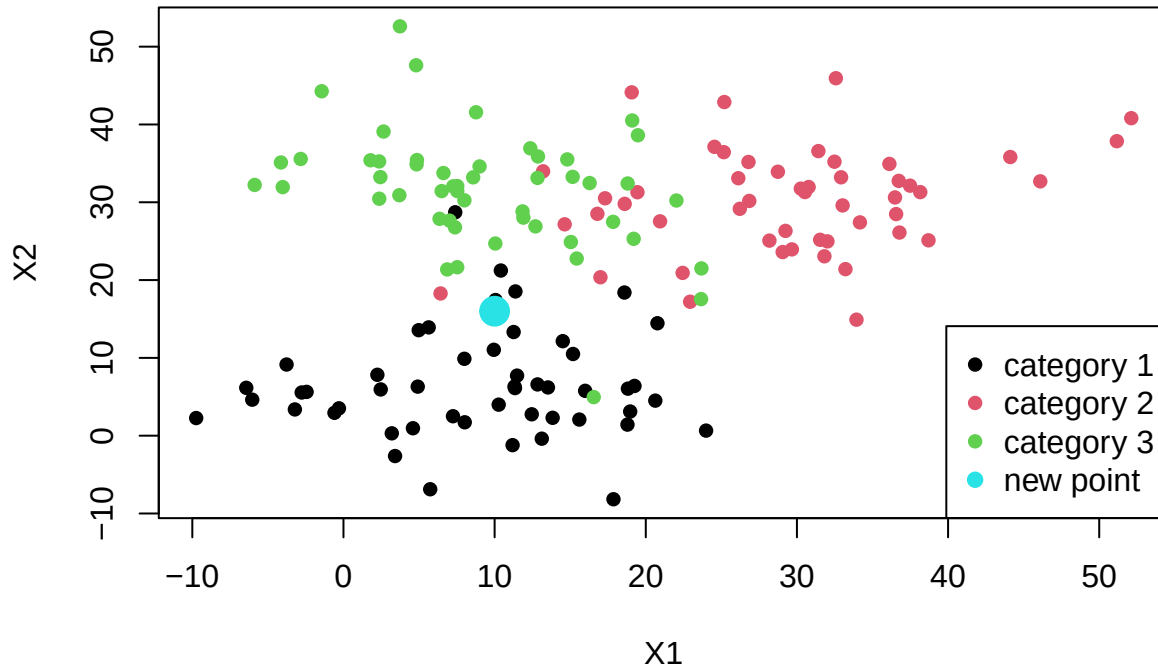


Next, consider a new point $X_t = (10, 16)$:

```
newpoint=c(10,16)
plot(dd[,1:2],col=dd[,3],xlab="X1",ylab="X2",main="Sample Data",pch=16)
points(newpoint[1],newpoint[2],pch=19,col=5,cex=2)

legend("bottomright",c("category 1","category 2","category 3","new point"),col=c(1:3,5),pch=c(16,16,16,
```

Sample Data



Next, we calculate the distance between each training data point and the new data point:

```
distvec=sqrt((newpoint[1]-dd[,1])^2+(newpoint[2]-dd[,2])^2)
```

Show the first 20 data points with their distance from newpoint:

```
cbind(dd,distvec)[1:20,]
```

```
##
##          distvec
## [1,] 23.9894580  0.6501089  1 20.768343
## [2,] 14.5087407 12.1489577  1  5.929525
## [3,] 18.5894221 18.4000452  1  8.918430
## [4,] 17.8618619 -8.1696963  1 25.416197
## [5,] 11.2572442 13.3239855  1  2.956639
## [6,]  5.6385884 13.9198921  1  4.832055
## [7,] -2.7639393  5.5480834  1 16.497294
## [8,] 15.9903699  5.7658079  1 11.858466
## [9,] -2.4393225  5.6277736  1 16.196290
## [10,] 18.9683464  3.1057966  1 15.706423
## [11,]  4.9140876  6.3036812  1 10.949206
## [12,] -9.7443999  2.2665319  1 24.050977
## [13,] 10.2692528  3.9871632  1 12.015854
## [14,] 15.1869594 10.4992959  1  7.560575
## [15,] 10.0589062 17.4322090  1  1.433420
## [16,] -3.7652369  9.1457322  1 15.377345
## [17,] 18.8196176  6.0417374  1 13.302355
## [18,] -0.5951438  2.9419107  1 16.815789
## [19,] -0.2987646  3.5152960  1 16.184325
## [20,]  7.3936929 28.7129788  1 12.977391
```


Now, find the K nearest neighbors and their category. We will use the `order` function. This gives the list of indices that will return a sorted list. For example:

```
set.seed(1000)
(a=runif(10,0,20)) #generate 10 random values from 0 to 20
```

```
## [1] 6.557574 15.176930 2.278728 13.815103 10.328048 1.354759 14.774297
## [8] 11.670701 4.315428 5.122448
```

```
sort(a) #sort
```

```
## [1] 1.354759 2.278728 4.315428 5.122448 6.557574 10.328048 11.670701
## [8] 13.815103 14.774297 15.176930
```

```
order(a) #indexes that produce the sort
```

```
## [1] 6 3 9 10 1 5 8 4 7 2
```

```
a[order(a)]
```

```
## [1] 1.354759 2.278728 4.315428 5.122448 6.557574 10.328048 11.670701
## [8] 13.815103 14.774297 15.176930
```

We use `order` rather than `sort` because we want to sort the category column according to the ordering of the distance values:

```
dd[order(distvec),3]
```

```
## [1] 1 1 1 2 1 1 1 1 1 3 3 1 1 2 1 3 3 1 1 1 3 1 1 1 3 3 1 1 3 1 2 3 3 1 3 3
## [38] 1 2 3 1 1 2 1 3 1 3 1 2 3 1 3 1 1 3 1 1 3 2 1 3 1 1 2 2 3 3 3 1 1 1 1 1 1
## [75] 3 3 3 2 3 3 2 1 3 3 3 3 1 3 1 1 3 3 3 2 2 3 1 2 3 3 2 3 2 2 3 2 1 3 2 2 3
## [112] 2 2 2 1 3 3 1 2 2 3 2 2 2 2 3 2 2 2 2 2 2 2 2 2 2 2 3 2 2 2 3 2 2 3 2 2 2
## [149] 2 2
```

Now, the category values are sorted by the corresponding distance from the target point. Next, we extract the categories of the top K closest points:

```
K=5
dd[order(distvec),3][1:K]
```

```
## [1] 1 1 1 2 1
```

The closest three points are category 1 (look at the figure), the fourth is category 2, and the fifth closest is category 1.

Next, we find the most frequent category. The `table` function can be used to count the number of occurrences of each category:

```
table(dd[order(distvec),3][1:K])
```

```
##  
## 1 2  
## 4 1
```

Category one occurs four times in the top five and category 2 appears once.

We can then use the `sort` function to sort.

```
sort(table(dd[order(distvec),3][1:K]), decreasing=T)
```

```
##  
## 1 2  
## 4 1
```

The first value after sorting is most frequent:

```
sort(table(dd[order(distvec),3][1:K]), decreasing=T)[1]
```

```
## 1  
## 4
```

Then, we use the `names` function to find the category name of the most frequent:

```
names(sort(table(dd[order(distvec),3][1:K]), decreasing=T)[1])
```

```
## [1] "1"
```

Exercise 4:

Using the code above as a template, write a function that does the KNN method:

```
my_knn(tdat,npoint,K)
```

The function should take input of the training data, the new point, and the value of K. It should output the predicted category.

knn function in R

The `knn` function does the K-nearest neighbors method. Here is an example with the stock market data.

We do the following:

1. Set the training data to be the data for 2001 to 2004 (i.e. <2005).
2. Define `Direction.train` to be the response for the training data.
3. Define `Direction.2005` to be the response for the test data (that is, the values that we want to predict).
4. Define `train.X` and `test.X` to be the training and test data for the predictors.

```
library(class)
library(ISLR)
attach(Smarket)
train =(Year <2005)

Direction=Smarket$Direction
Direction.2005=Direction[!train]
Direction.train=Direction[train]

train.X=cbind(Lag1,Lag2)[train,]
test.X=cbind(Lag1,Lag2)[!train,]
```

Now, run the `knn` function. It takes input of the training data for the predictors, the the test data for the predictors, and the true classifications for the training data.

```
set.seed(1) # if several observations are tied as nearest neighbors, then R will randomly break the tie
knn.predk1=knn(train.X,test.X,Direction.train,k=1)
```

The output from `knn` is the predicted categories for the test data:

```
knn.predk1[1:30]
```

```
## [1] Up   Down Up   Up   Up   Down Down Down Down Up   Up   Up   Up   Up   Down
## [16] Down Up   Up   Up   Up   Down Down Down Down Down Up   Up   Up   Down Up
## Levels: Down Up
```

Make a table of the predicted versus actual test values:

```
table(knn.predk1,Direction.2005)
```

```
##           Direction.2005
## knn.predk1 Down Up
##           Down   43 58
##           Up    68 83
```

```
mean(knn.predk1==Direction.2005)
```

```
## [1] 0.5
```

Repeat with `k=3`:

```
knn.predk3=knn(train.X,test.X,Direction.train,k=3)
table(knn.predk3,Direction.2005)
```

```
##           Direction.2005
## knn.predk3 Down Up
##           Down   48 54
##           Up    63 87
```

```
mean(knn.predk3==Direction.2005)
```

```
## [1] 0.5357143
```

It does slightly better with $k=3$ (but this may well just be chance).